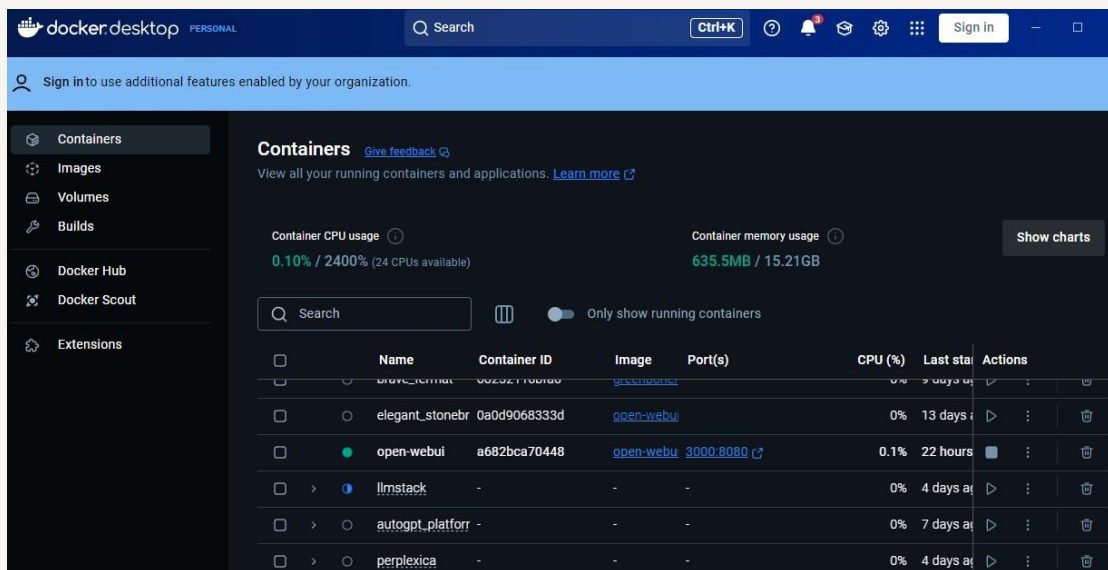


Deploy an App with Docker

Ori Mizrahi



Introducing Today's Project!

What is Docker?

Docker is a tool that packages apps into containers, ensuring they run the same everywhere. In today's project, I used Docker to create a container with a Dockerfile. This made deployment easy and consistent, saving time and reducing errors.

One thing I didn't expect...

One thing i didn't expect was seeing how easy and quickly us to deploy an application using Elastic Beanstalk.

This project took me...

This project took to me 2.5 hours, including all the demo time.

Understanding Containers and Docker

Containers

Containers are tools for packaging applications in a way that is easy for developers to run. They are useful because it help developes and engineeres together in a team to share their work in a more efficient way.

A container image is like a snapshot of an app, capturing all the bits and pieces it needs to run smoothly. Think of it as a self-contained package that includes the code, libraries, and settings. Great for developers and engineers.

Docker

Docker is a platform for creating and managing containers. Docker make working with containers easy. Docker Desktop is a software for use docker.

The Docker daemon is a background service that manages Docker containers, images, and networks. It runs on the host machine and handles requests from the Docker CLI or API, enabling the creation, running, and stopping of containers.

Running an Nginx Image

Nginx is a web server and reverse proxy that's widely used for its high performance and scalability. It handles multiple client requests simultaneously, it great for busy websites. Plus, it can act as a reverse proxy, load balancer and etc.

The command to start a new container was 'docker run'. We also set the flags '-d p 80:80 nginx', which means we're running the container in the background (-d) and we're matching the port 80 in our host computer to the container's port 80 (-p 80:80)

Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

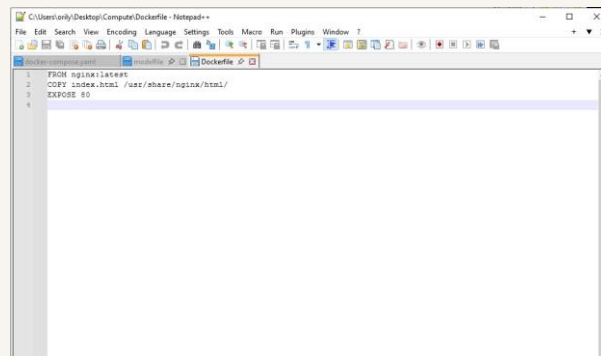
Thank you for using [nginx](#).

Creating a Custom Image

The Dockerfile is set of instructions that tells Docker how to build your custom container image.

Our Dockerfile tells Docker three things. First, our custom container image uses the latest version of the NGINX container image at its base. Then modifying this base by replacing the default Nginx welcome page with our own custom index.html.

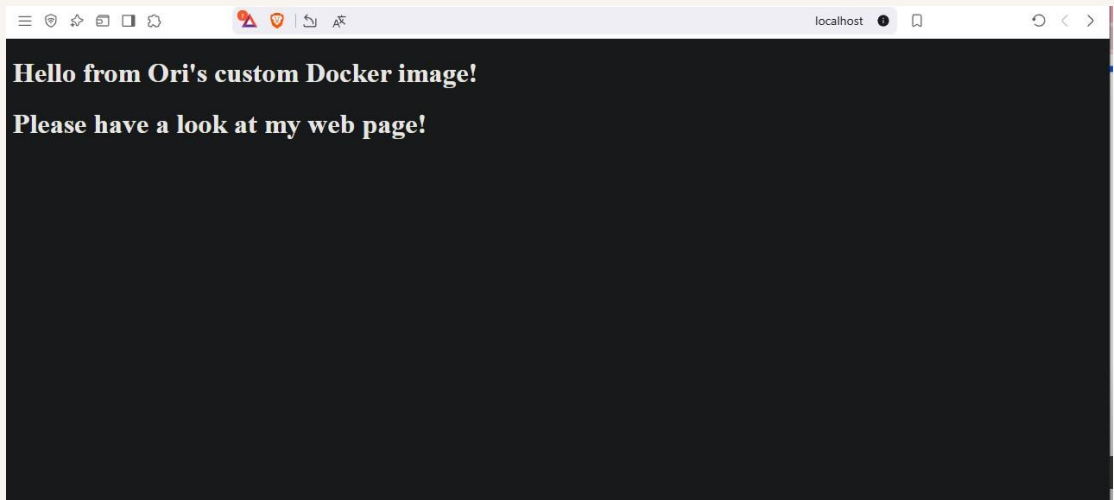
The command we used to build a custom image with our Dockerfile was 'docker build'. The '.' at the end of the command means that Docker can find the Docker file in the current directory i.e. the Compute folder on our Desktop.



Running My Custom Image

There was an error when we ran our custom image because we tried to map out port 80 with the new container's port 80 but a running container was already port 80. We resolved this by stopping the running container so that we can start new one.

In this example, the container image is the template for creating a new container running an NGINX server that serves our custom 'index.html' file. The container is the actual software that's running an NGINX server with those customization.



Elastic Beanstalk

Elastic Beanstalk is an AWS service that helps with deploying application in the cloud. It abstracts away a lot of the work with setting up cloud infrastructure when deploying applications

Deploying our custom image with Elastic Beanstalk took us 10 minutes. This includes the time it took to launch the Elastic Beanstalk application.

